

Bank Customer Segmentation & Term Deposit Subscription Prediction

A Data Science Project Report

Domain:	Banking & Finance
Dataset:	UCI Bank Marketing (41,188 rows)
Tools:	Python, Scikit-learn, Dash
Date:	May 2026
AI Assistance:	Claude (Anthropic)

1. Executive Summary

This project applies supervised machine learning and unsupervised clustering to the UCI Bank Marketing dataset to solve two interconnected business problems: predicting which customers are likely to subscribe to a term deposit, and segmenting customers into actionable groups for targeted marketing.

The analysis covers 41,188 customer records from a Portuguese bank's direct marketing campaigns. Three classification models were developed and evaluated — Logistic Regression, Random Forest, and Decision Tree — alongside a KMeans clustering analysis. Results were visualised through two interactive dashboards built in Streamlit and Dash by Plotly, enabling non-technical stakeholders to explore insights without any coding knowledge.

Key Results

Subscription rate: 11.3% | Best model AUC: ~0.800 (Logistic Regression) | Best performing segment: Senior customers (45.5% subscription rate) | Strongest predictor: Previous campaign outcome (65.1% re-subscription after success)

2. Problem Statement

Banks run direct marketing campaigns to sell financial products such as term deposits. These campaigns involve phone calls to customers, which are expensive in both time and resources. Without intelligent targeting, campaign managers contact a large number of customers who have little to no intention of subscribing, leading to wasted resources and a poor customer experience.

This project addresses two specific questions:

- **Q1.** Can we predict, before making a call, whether a customer is likely to subscribe to a term deposit?
- **Q2.** Can we group customers into meaningful segments to enable more personalised, efficient marketing strategies?

Answering these questions enables the bank to prioritise high-potential customers, reduce unnecessary contacts, improve conversion rates, and make better use of campaign budgets.

3. Dataset Description

The dataset used is the UCI Bank Marketing Dataset, one of the most widely used benchmark datasets in marketing analytics. It records outcomes from direct marketing campaigns (phone calls) conducted by a Portuguese banking institution.

Source	UCI Machine Learning Repository
Records	41,188 customer interactions
Features	20 input variables + 1 target variable
Target (y)	Binary: 'yes' (subscribed) / 'no' (did not subscribe)
Class balance	~88.7% No, ~11.3% Yes — heavily imbalanced
Time period	May 2008 to November 2010

Feature Categories

- Client demographics: age, job, marital status, education, credit default, housing loan, personal loan
- Last contact: contact type (cellular/telephone), month, day of week
- Campaign data: number of contacts, days since last contact (pdays), previous contacts
- Previous outcome: result of prior campaign (success, failure, nonexistent)
- Economic context: employment variation rate, consumer price index, consumer confidence index, euribor 3-month rate, number of employees

Important Note on Duration

The 'duration' variable (call duration in seconds) was deliberately excluded from the model. Although it has very high predictive power, it is not available before a call is made — including it would constitute data leakage and make the model useless for real-world deployment. This is a non-obvious but critical design decision.

4. Methodology

4.1 Data Preprocessing

Several preprocessing steps were applied before modelling:

- "unknown" string values were replaced with NaN (missing value) rather than treated as a valid category, since 'unknown' carries no predictive signal.
- day's value of 999 (sentinel value meaning 'never contacted') was recorded to -1 to distinguish it from actual day counts.
- The duration column was dropped to prevent data leakage.

- All categorical variables were one-hot encoded using `pd.get_dummies(drop_first=True)`, producing binary indicator columns for each category level.
- Numeric features were standardised using `StandardScaler` before Logistic Regression training, as LR is sensitive to feature scale.

4.2 Handling Class Imbalance

The dataset is heavily imbalanced — only 11.3% of customers subscribed. This is a critical challenge: a naive model that always predicts 'No' would achieve ~89% accuracy while being completely useless. Three strategies were applied:

- `class_weight='balanced'` was set in all classifiers, which automatically adjusts sample weights inversely proportional to class frequency.
- `stratify=y` was used in `train_test_split` to preserve the class ratio in both training and test sets.
- ROC-AUC was used as the primary evaluation metric instead of accuracy, since AUC measures ranking quality independent of class balance.

4.3 Classification Models

Three supervised models were trained and compared:

Model	Key Parameters	Strengths
Logistic Regression	<code>max_iter=1000</code> , <code>class_weight=balanced</code>	Interpretable, good recall, best AUC
Random Forest	<code>n_estimators=100</code> , <code>class_weight=balanced</code>	Best F1, robust, feature importance
Decision Tree	<code>max_depth=8</code> , <code>min_samples_leaf=20</code> , <code>class_weight=balanced</code>	Explainable rules, fast inference

The Decision Tree was pruned (`max_depth=8`, `min_samples_leaf=20`) to prevent overfitting. Without pruning, the tree memorises training data and generalises poorly to new customers.

4.4 Customer Segmentation

KMeans clustering was applied to group customers into behavioural segments. Unlike classification, clustering is unsupervised — it finds natural groupings without using the subscription label. Features used for clustering included age, campaign contacts, previous contacts, days, euriborm rate, employment variation rate, and number of employees. Features were standardized before clustering.

The optimal number of clusters (k=3) was determined using the elbow method, which plots within-cluster sum of squares (WCSS) against k and identifies the point of diminishing returns. Subscription rates were then calculated per cluster to connect the unsupervised segments back to the business outcome.

5. Results & Key Findings

5.1 Model Performance

Model	ROC-AUC	Recall (Yes)	F1 (Yes)
Logistic Regression	~0.800	~0.36	~0.333
Random Forest	~0.778	~0.31	~0.399
Decision Tree	~0.800	~0.25	~0.366

Recommended Model

Logistic Regression is recommended when the goal is to identify as many potential subscribers as possible (maximising recall). Random Forest is recommended for production use due to its highest F1 score and robustness. ROC-AUC should always be the primary comparison metric for this imbalanced problem.

5.2 Key Business Insights

Customer Demographics

- Senior customers (60+) have the highest subscription rate at approximately 45.5% — more than 4x the overall average.
- Young customers (under 25, likely students) also over-index at around 20.9%.

- University degree holders subscribe at a higher rate (~13.7%) than lower education groups.
- Single customers subscribe more frequently (~14%) than married or divorced customers.

Campaign Effectiveness

- Cellular contact converts at approximately 14.7% vs. 5.2% for telephone — nearly 3x more effective. All campaigns should prioritise cellular outreach.
- Previous campaign success is the single strongest predictor: customers who subscribed before have a ~65% re-subscription rate. These should be contacted first.
- Campaign frequency shows strong diminishing returns after 3 contacts. Contacting customers 6 or more times yields very low returns and risks opt-outs.

Customer Segments

- Cluster 0 (High-Value): Moderate age, prior engagement history, moderate campaign contacts. Most likely to convert. Priority segment for direct campaigns.
- Cluster 1 (Nurture): Minimal interaction history. Requires awareness-building and relationship development before a conversion push.
- Cluster 2 (Over-Targeted): High campaign contact count, low historical response rate. Marketing spend should be reduced on this segment and messaging redesigned.

6. Interactive Dashboards

Two fully functional interactive dashboards were developed to make the project findings accessible to non-technical stakeholders. Both dashboards accept CSV file uploads and dynamically update all charts and model metrics based on the uploaded data.

6.1 Streamlit Dashboard

Built using the Streamlit framework, the Streamlit dashboard is designed for data scientists and analysts who want to quickly explore findings. It features six pages:

- Overview: KPI metrics, subscription split, job-level conversion rates, and monthly trend chart
- Demographics: Age group analysis, distribution histogram, marital status, education breakdown
- Financial Behavior: Balance distribution, housing/personal loan impact, economic indicator violin plots
- Campaign Analysis: Contact method effectiveness, previous outcome impact, campaign frequency analysis
- ML Models: Side-by-side model comparison table, ROC curves, feature importance chart, confusion matrices, and a live prediction simulator

- Customer Segments: Elbow method chart, cluster scatter plot, cluster profile table, subscription rate per cluster

Deployment: Streamlit dashboards can be deployed for free on Streamlit Cloud (share.streamlit.io) with a public URL accessible to anyone.

6.2 Dash by Plotly Dashboard

Built using Dash by Plotly with Bootstrap components, the Dash dashboard has a more production-grade web application appearance, with a dark sidebar navigation, card-based layout, and fully interactive Plotly charts. It covers the same six pages as the Streamlit version.

Deployment: Dash applications can be deployed free on Render.com or Railway.app using the included Procfile. The server object (`app.server`) exposes a WSGI interface served by Gunicorn for production use.

Technical Note on Dashboard Consistency

An important challenge encountered was ensuring dashboard model metrics matched notebook outputs. The initial dashboard implementation used `LabelEncoder` (integer encoding) for categorical variables, while the notebook used `pd.get_dummies(drop_first=True)` (one-hot encoding). These produce fundamentally different feature matrices, causing AUC score differences of 0.01-0.03. After aligning both pipelines to use `get_dummies`, results became consistent. Minor residual differences (< 0.003) remain due to DataFrame JSON serialisation precision in Dash's `dcc.Store` component.

7. AI Assistance — Working with Claude

Throughout this project, I used Claude (Anthropic) as an AI assistant to support development, debugging, and documentation. I want to be transparent about this, as I believe honest disclosure of AI tool usage is important in the current data science landscape.

7.1 How Claude Was Used

- Code review and bug identification: Claude reviewed the Jupyter notebook and identified two critical variable naming bugs where model predictions were passed to the wrong accuracy calculation cells, producing incorrect metric comparisons. It also identified a missing `random_state` parameter in `train_test_split` causing non-reproducible results.
- Improvement suggestions: Claude suggested adding `class_weight='balanced'` to handle class imbalance, pruning the Decision Tree with `max_depth` and `min_samples_leaf`, adding ROC-AUC as a primary metric, expanding clustering features from 3 to 7 variables, and adding feature importance charts and confusion matrices.
- Dashboard development: Claude generated both the Streamlit and Dash dashboard codebases from scratch, based on specifications I provided about the project scope and desired visualisations.

- Debugging pipeline mismatches: When model metrics differed between the notebook and dashboards, Claude diagnosed the root cause (LabelEncoder vs get_dummies encoding mismatch) and fixed both dashboard codebases.
- Report writing: Claude assisted in drafting this project report based on my project description and findings.

7.2 What I Contributed

- The original data analysis, EDA structure, and business interpretation in the notebook were my own work.
- All decisions about which models to use, which features to include, and what business conclusions to draw were mine.
- I reviewed, tested, and validated every code change Claude suggested before accepting it.
- The project direction, scope, and framing were entirely self-driven.

Reflection on AI-Assisted Development

Using Claude accelerated the development process significantly, particularly for boilerplate dashboard code and systematic code review. However, it required critical evaluation at every step — Claude's suggestions are not always correct, and several required adaptations to fit the specific project context. The most valuable use was as a second pair of eyes on the code, catching issues like the variable naming bugs that are easy to miss during self-review.

8. Limitations & Future Work

8.1 Current Limitations

- The recall for the 'Yes' class remains relatively low (~0.25-0.36 depending on model). This means the model still misses a significant portion of actual subscribers.
- Clustering interpretability could be improved with SHAP values or cluster centroid profiling against domain knowledge.
- The project does not include hyperparameter tuning (GridSearchCV or Optuna) which could meaningfully improve model performance.
- No deployment pipeline exists — the models are retrained each time the dashboard loads, which is inefficient at scale.

8.2 Planned Next Steps

- Power BI Dashboard: A Power BI version of the dashboard is planned for development, targeting business analyst and non-technical stakeholder audiences in enterprise settings.
- SMOTE oversampling: Apply Synthetic Minority Oversampling Technique as an alternative to class_weight for handling imbalance.

- Hyperparameter tuning: Use cross-validated grid search to optimise model parameters systematically.
- Model export: Export trained models and metrics to JSON/pickle files from the notebook, eliminating dashboard retraining and ensuring exact metric consistency.
- XGBoost / LightGBM: Evaluate gradient boosted tree models which typically outperform Random Forest on tabular classification tasks.

9. Technology Stack

Python 3.11	Core programming language
Pandas / NumPy	Data manipulation and numerical computing
Scikit-learn	Machine learning models, preprocessing, evaluation metrics
Plotly	Interactive visualisation library used in both dashboards
Streamlit	Rapid dashboard framework for data science applications
Dash by Plotly	Production-grade Python web application framework
Dash Bootstrap Components	UI component library for Dash (cards, grid, tables)
SQLite3 (in-memory)	SQL analysis layer within the notebook
Jupyter Notebook	Interactive development and analysis environment
GitHub	Version control and public project sharing
Claude (Anthropic)	AI assistant for code review, debugging, and documentation

10. Conclusion

This project demonstrates a complete end-to-end data science workflow applied to a real-world banking problem. Starting from raw customer data, it progresses through cleaning, exploratory analysis, feature engineering, model training, evaluation, and final delivery as interactive business intelligence dashboards.

The core finding is that term deposit subscription is predictable from customer and campaign features, with ROC-AUC scores of approximately 0.800. The most actionable insights are: prioritise re-contacting previously successful customers, use cellular contact over telephone, target senior and student segments, and limit campaign contacts to three per customer.

The customer segmentation provides a complementary strategic layer, dividing the customer base into high-value conversion targets, nurture-stage prospects, and over-contacted customers requiring a reduced-frequency approach.

The project is ongoing — Power BI dashboard development is the next planned milestone, followed by model optimisation and deployment infrastructure. All project files are available on GitHub.

*This report was produced with assistance from Claude (Anthropic). **Power BI dashboard coming soon.***